Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ***ИУК «Информатика и управление»***

КАФЕДРА ***ИУК5 «Системы обработки информации»***

Лабораторная работа № 6

«Ансамбли решающих деревьев: оценка значимости признаков»

ДИСЦИПЛИНА: «Технологии обработки больших данных»

Выполнил: студент гр.ИУК5-82Б _____ (Ли Р.В. _____)
(Подпись) (Ф.И.О.)

Проверил: _____ (_____)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

Калуга, 2026

Цель лабораторной работы - освоить методы оценки значимости признаков для моделей на основе ансамблей решающих деревьев (bagging/boosting/stacking/voting) и научиться интерпретировать вклад признаков с помощью встроенных важностей, SHAP-значений и последовательного отбора признаков.

Задачи:

- Загрузить табличный датасет, выполнить базовый анализ и предобработку данных.
- Разделить данные на обучающую и тестовую выборки, выбрать метрики качества для бинарной классификации (например, F1 и PR AUC).
- Обучить и сравнить несколько моделей: базовую (логистическая регрессия) и ансамбли решающих деревьев (например, RandomForest, AdaBoost, XGBoost, Voting/Stacking).
- Получить и визуализировать важность признаков разными способами:
    - коэффициенты в логистической регрессии;
    - встроенная важность признаков у моделей на деревьях;
    - SHAP-значения для моделей на деревьях и их визуализации (summary/dependence/force).
- Выполнить последовательный отбор признаков (forward-backward/SequentialFeatureSelector) и сравнить наборы «лучших» признаков с результатами встроенной важности и SHAP.
- Проверить влияние отбора признаков: обучить модели на топ-N признаках и на «наименее важных» признаках и сравнить качество.

Ход работы:

[2]:
```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, VotingClassifier

from sklearn.metrics import f1_score, average_precision_score

from sklearn.feature_selection import SequentialFeatureSelector

import shap
import matplotlib.pyplot as plt
import seaborn as sns
```

[3]:
```
df = pd.read_csv("diabetes.csv")
df.head()
```

[3]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

[4]:
```
df.info()
df.describe()

df.isnull().sum()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

[4]:
```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

[5]:
```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state=42, stratify = y
)
```

[6]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[7]:
```
log_reg = LogisticRegression(max_iter = 1000)
log_reg.fit(X_train_scaled, y_train)
```

[7]:

▾ LogisticRegression

▸ Parameters

[8]:
```
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
```

[8]:

▾ RandomForestClassifier

▸ Parameters

[9]:
```
ada = AdaBoostClassifier(random_state=42)
ada.fit(X_train, y_train)
```

[9]:

▾ AdaBoostClassifier

▸ Parameters

[10]:
```
voting = VotingClassifier(
    estimators = [
        ("lr", log_reg),
        ("rf", rf),
        ("ada", ada)
    ],
    voting = "soft"
)

voting.fit(X_train_scaled, y_train)
```

[10]:

▸ VotingClassifier

lr: ▸ LogisticRegression

rf: ▸ RandomForestClassifier

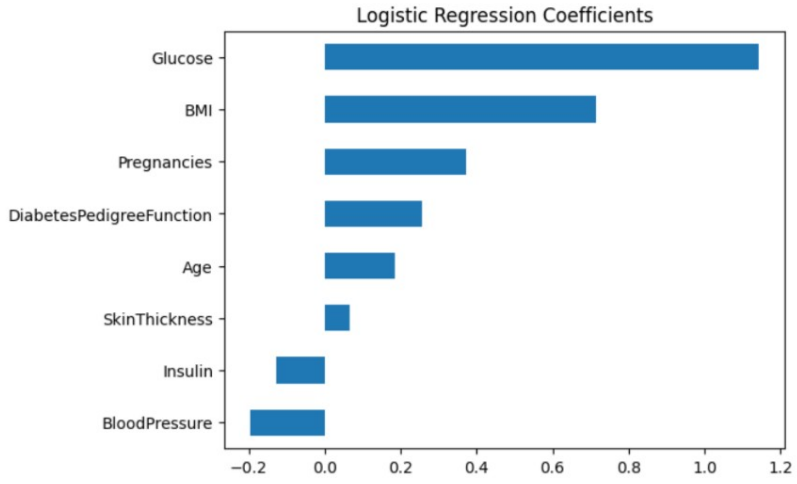
ada: ▸ AdaBoostClassifier

[11]:
```
def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:,1]
    f1 = f1_score(y_test, y_pred)
    pr_auc = average_precision_score(y_test, y_proba)
    return f1, pr_auc
```

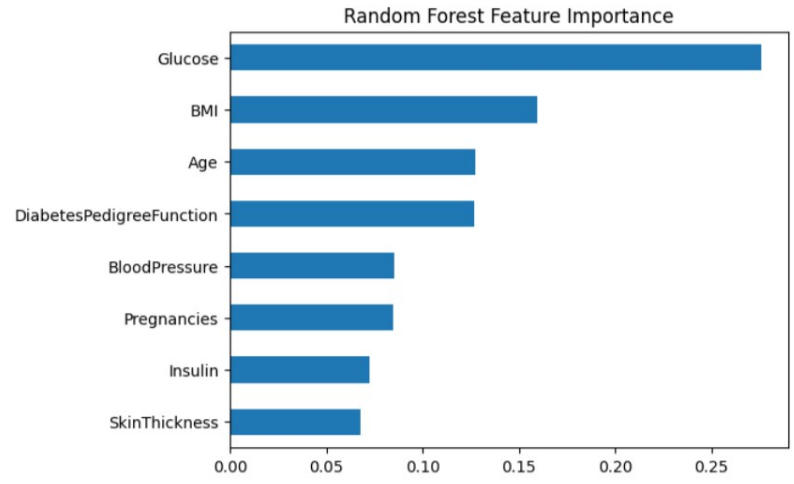
[12]:
```
print("LogReg:", evaluate(log_reg, X_test_scaled, y_test))
print("RF:", evaluate(rf, X_test, y_test))
print("Ada:", evaluate(ada, X_test, y_test))
print("Voting:", evaluate(voting, X_test_scaled, y_test))
```

```
LogReg: (0.56, 0.6922804772647576)
RF: (0.6336633663366337, 0.7005073870983374)
Ada: (0.6730769230769231, 0.6864237948278183)
Voting: (0.5918367346938775, 0.7100710054282928)
```

[13]:
```
coef = pd.Series(log_reg.coef_[0], index=X.columns)
coef.sort_values().plot(kind="barh")
plt.title("Logistic Regression Coefficients")
plt.show()
```



[14]:
```
importances = pd.Series(rf.feature_importances_, index=X.columns)
importances.sort_values().plot(kind="barh")
plt.title("Random Forest Feature Importance")
plt.show()
```

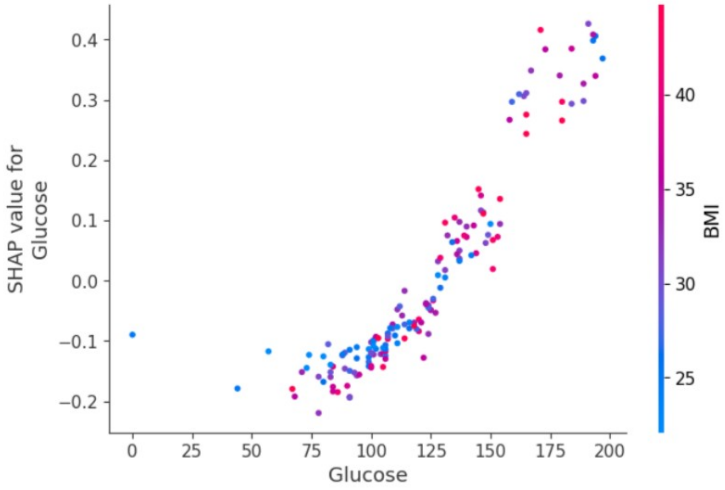


[15]:
```
explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test)
```

[16]:
```
shap.summary_plot(shap_values, X_test)
```



[17]:
```
shap.dependence_plot("Glucose", shap_values[:,:,1], X_test)
```



```
[28]: sfs = SequentialFeatureSelector(
          rf,
          n_features_to_select=5,
          direction="forward",
          scoring="f1",
          cv=5
      )

      sfs.fit(X_train, y_train)
```

[28]:
▸ SequentialFeatureSelector
▸ estimator: RandomForestClassifier
▸ RandomForestClassifier

```
[29]: selected_features = X.columns[sfs.get_support()]
      selected_features

[29]: Index(['Pregnancies', 'Glucose', 'BMI', 'DiabetesPedigreeFunction', 'Age'], dtype='str')

[30]: X_train_sel = X_train[selected_features]
      X_test_sel = X_test[selected_features]

      rf_sel = RandomForestClassifier(random_state=42)
      rf_sel.fit(X_train_sel, y_train)

      evaluate(rf_sel, X_test_sel, y_test)

[30]: (0.6274509803921569, 0.6709990422510004)
```

```
[31]: worst_features = importances.sort_values().head(5).index

      X_train_worst = X_train[worst_features]
      X_test_worst = X_test[worst_features]

      rf_worst = RandomForestClassifier(random_state=42)
      rf_worst.fit(X_train_worst, y_train)

      evaluate(rf_worst, X_test_worst, y_test)

[31]: (0.4583333333333333, 0.5397930920350684)
```

Вывод: были освоены методы оценки значимости признаков для моделей на основе ансамблей решающих деревьев (bagging/boosting/stacking/voting) и научиться интерпретировать вклад признаков с помощью встроенных важностей, SHAP-значений и последовательного отбора признаков.